
GEN REPORT

/todo(title)

/todo(subtitle)

datalab@unimib.it

Abstract

/todo(abstract)

...

Keywords: documentation, template, pandoc, latex

Contents

TOC	2
Q:1	3
Q:1 - /todo(q1-title)	3
Raw LaTeX embedding example	3
pandoc YAML front matter processing issue for gfm input format	4
System Hints	5
Question Prompt	6
Response Template	7
A:1 (Claude)	8
A:1 (Gemini)	9
A:1 (ChatGPT)	10
A:1 (Perplexity)	11
Q:1.2 (Perplexity)	11
A:1.2 (Perplexity)	11
A:1 (DeepSeek)	12

TOC

1. Q:1 - /todo(q1-ref)
 - see: /todo(a1-ref-claude) (Claude)
 - see: /todo(a1-ref-gemini) (Gemini)
 - see: /todo(a1-ref-chatgpt) (ChatGPT)
 - see: /todo(a1-ref-perplexity) (Perplexity)
 - see: /todo(a1-ref-deepseek) (DeepSeek)

Q:1

Q:1 - /todo(q1-title)

| ^ |

/todo(q1-intro) ...

Raw LaTeX embedding example

By default, Pandoc will preserve raw LaTeX code in Markdown documents when converting the document to LaTeX, so you can use LaTeX commands or environments in Markdown. However, sometimes your LaTeX code might be too complex for Pandoc to parse, in which case Pandoc will treat the content as normal Markdown. The consequence is that special LaTeX characters may be escaped, e.g., Pandoc may convert a backslash `\` to `\\textbackslash{}`.

To make sure that Pandoc does not touch the raw LaTeX code in your Markdown document, you may wrap the code in a fenced code block with the attribute `=latex`, e.g.,

```
A B
A B
```

Do not forget the equal sign before `latex`, i.e., it is `=latex` instead of `latex`. This feature requires a Pandoc version higher than 2.0 (check `rmarkdown::pandoc_version()`).

```
test inline ++ aaa ++, // ⇒ → ≥ ≤ ≠
```

```
test quote ++ aaa ++, // ⇒ → ≥ ≤ ≠
```

```
// test ligatures
// ++ ++ -- || &&
// ⇒ → ≥ ≤ ≠
```

```
int main(char[] argv) {
    return 0;
}
```

read the f*g [manual](#)...

as someone said¹

- [rtfm-1]: aaa -> [Wikipedia:RTFM](#)

¹RTFM [rtfm-1]

pandoc YAML front matter processing issue for gfm input format

With pandoc, converting a markdown file to pdf show a different behavior on treatment of YAML front matters depending on the input format specified:

1. markdown: YAML preamble is omitted by the final output, as desired, but "GitHub Flavored Markdown (GFM)" option are not active.
2. gfm+yaml_metadata_block: the pdf is produced with GFM options enabled, as requested, but with an unwanted effect: YAML preamble is included in final pdf and the initial "Table of Contents" is suppressed
3. gfm: with this option, that should indicate exactly what is requested: GFM markdown enabled, YAML omitted and Table of Content included, the generation failed, with no pdf generation

In case 3, the problem is related to the wrong interpretation of this part of YAML preamble:

```

---
# omitted ...

header-includes:
- |
  ````{=latex}
 \input{../_styles/_md-report_.tex}
 ...

omitted ...

```

This gets translated in LaTeX markup instead of verbatim inclusion in TeX source, passed to LuaLaTeX pdf engine.

```

header-includes:

\begin{itemize}
\tightlist
\item
 \textbar{}

\begin{Shaded}
\begin{Highlighting}[]
\NormalTok{\textbackslash}\input\{../_styles/_md{-}report_tex\}
\end{Highlighting}
\end{Shaded}
\end{itemize}

```

This is likely a bug in gfm format processing. How to resolve the issue, even with some workaround?

## System Hints

[!tip]

/todo(q1-system)

As ...

Your task ...

Reply in clear formatted "GitLab Flavored Markdown (GLFM)" markdown, with precise (lint) validation (codeblock delimiters "" placed at line start).

At the end, provide, as Markdown footnotes, a list of references to online documentation resources, linked to answer as citation where appropriate.

Add any additional important information not explicitly required in an "Additional Notes" section

## **Question Prompt**

/todo(q1-prompt) ...

## Response Template

/todo(q1-template) ...

## Overview

## Details

\\\\`bash

#!/bin/bash

echo "\$(date -isec) - (rc=\${rc:-\$?}) completed."

\\\\`

## A:1 (Claude)

| ^ | /*todo(a1-ref-claude)*

/todo(a1-claude) ...

## A:1 (Gemini)

| ^ | /*todo(a1-ref-gemini)*

/todo(a1-gemini) ...

## A:1 (ChatGPT)

| ^ | /*todo(a1-ref-chatgpt)*

/todo(a1-chatgpt) ...

## A:1 (Perplexity)

| ^ | /*todo(a1-ref-perplexity)*

/todo(a1-perplexity) ...

### Q:1.2 (Perplexity)

| ^ | (*=> continue*)

/todo(q1.2-perplexity) ...

---

### A:1.2 (Perplexity)

| ^ | (*=> continue*)

/todo(a1.2-perplexity) ...

## A:1 (DeepSeek)

| ^ | /*todo(a1-ref-deepseek)*

/todo(a1-deepseek) ...